

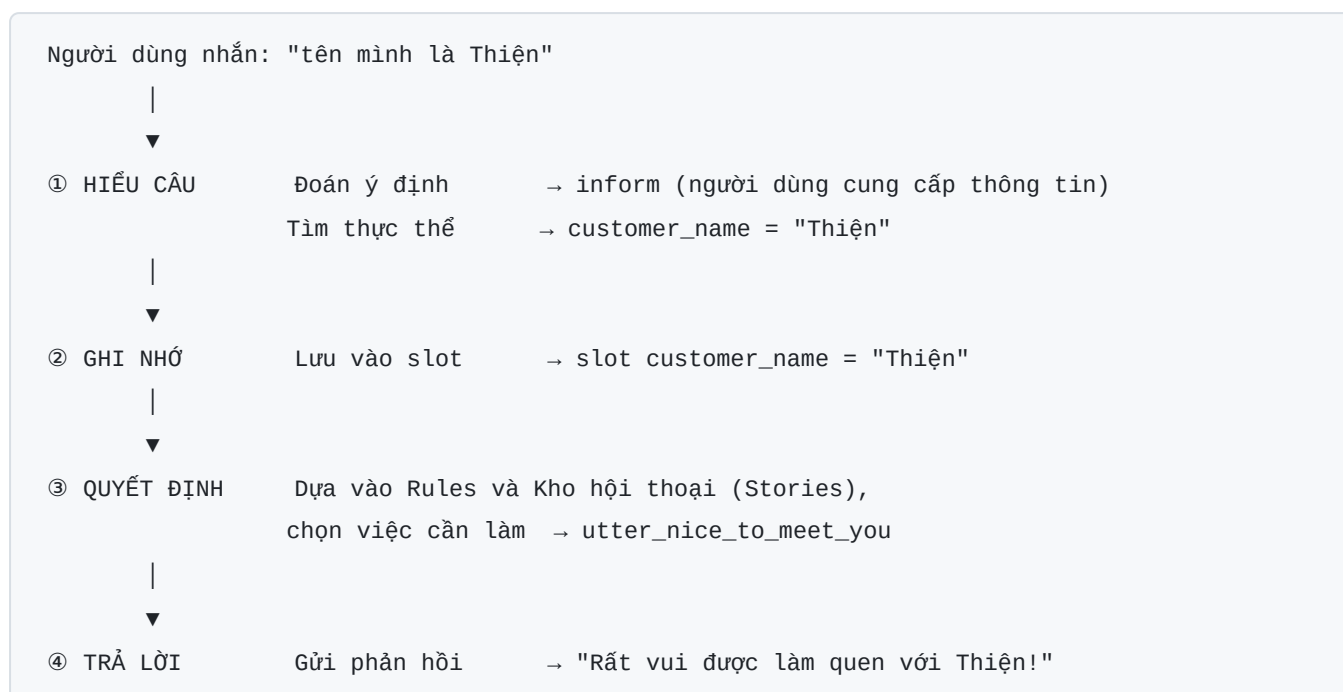
Hướng dẫn thiết kế chatbot trên trang quản trị

Tài liệu này dành cho người thiết kế nội dung chatbot trên web: người nhập ý định, câu mẫu, câu trả lời và kịch bản hội thoại. Bạn không cần biết lập trình.

Các ví dụ lấy từ dữ liệu mẫu có sẵn: **AceBot**, trợ lý ảo chăm sóc khách hàng.

1. Bot hiểu và trả lời một tin nhắn như thế nào

Bot chạy trên nền **Rasa**. Mỗi khi người dùng nhấn một câu, bot làm 4 bước:



Mỗi trang trên web phục vụ một bước:

Bước	Trang trên web	Bạn nhập gì
① Hiểu câu	Ý định, Nlu, Thực thể	Các loại câu hỏi và câu mẫu cho từng loại
② Ghi nhớ	Slots	Những thông tin bot cần nhớ trong cuộc trò chuyện
③ Quyết định	Rules, Kho hội thoại	Nghe câu gì thì làm gì
④ Trả lời	Phản hồi	Nội dung bot nói ra

Nhập xong dữ liệu, bạn phải **Train** thì bot mới học được. Chưa train thì bot vẫn chạy theo dữ liệu cũ.

2. Làm một chức năng mới: 6 bước

Ví dụ: bot hay chào tạm biệt quá sớm, người dùng phản đối “*tôi chưa hỏi mà*”. Ta muốn bot xin lỗi và mời người dùng hỏi tiếp.

#	Làm gì	Ở trang	Ví dụ
1	Tạo ý định	Ý định	Mã <code>not_finished</code> , tên tiếng Việt “Người dùng chưa xong, muốn hỏi tiếp”
2	Nhập câu mẫu (tối thiểu 2 câu, nên 8–15 câu)	Nlu	“tôi chưa hỏi mà”, “khoan đã”, “đừng đi vội”, “mình còn muốn hỏi nữa”...
3	Tạo câu trả lời	Phản hồi	<code>utter_continue</code> : “Dạ xin lỗi bạn, mình vẫn ở đây. Bạn cứ hỏi tiếp nhé!”
4	Nối câu hỏi với câu trả lời	Rules	<code>not_finished</code> → <code>utter_continue</code>
5	Train	Train Model	Bấm Kiểm tra dữ liệu , rồi bấm Train
6	Thử lại	Chat thử	Gõ “khoan đã, mình còn câu hỏi” và xem bot hiểu có đúng không

Hay quên nhất là bước 4. Có ý định và câu trả lời nhưng chưa có rule hoặc story nào nối hai cái lại thì bot hiểu câu hỏi nhưng không biết phải trả lời gì.

3. Nhóm Dialogue: dữ liệu của bot

3.1. Ý định (Intent)

Dùng để làm gì: gom những câu cùng một mục đích thành một nhóm. Người dùng có thể hỏi “mấy giờ mở cửa”, “bên bạn làm việc giờ nào”, “thứ 7 có làm không”. Cách hỏi khác nhau nhưng cùng một ý định là `ask_working_hours`, và bot chỉ cần một câu trả lời cho cả nhóm.

Các ô cần nhập:

Ô	Ý nghĩa	Ví dụ
Mã ý định	Tên kỹ thuật dùng khi train. Viết thường, không dấu, nối bằng <code>_</code> . Không đổi sau khi đã dùng trong Rules/Stories.	<code>ask_working_hours</code>
Tên tiếng Việt	Tên dễ hiểu, hiện ra khi chọn ý định ở các trang khác	Hỏi giờ làm việc

Cách đặt mã dễ nhớ:

- `ask_...` khi người dùng hỏi: `ask_address`, `ask_bot_name`
- `request_...` khi người dùng muốn bot làm gì đó: `request_consultation`

- Các ý định dùng chung: `greet` (chào), `goodbye` (tạm biệt), `affirm` (đồng ý), `deny` (từ chối), `thank` (cảm ơn)

Lưu ý:

- Mỗi ý định chỉ nên có **một mục đích rõ ràng**. Nếu hai ý định có câu mẫu gần giống nhau, bot sẽ hay nhầm lẫn giữa chúng.
- Câu mẫu dùng để train được nhập ở trang **Nlu** (mục 3.2). Ở câu mẫu trên trang Ý định hiện **không được dùng khi train**.

3.2. Nlu: câu mẫu để bot học

Dùng để làm gì: đây là phần quan trọng nhất để bot *hiểu đúng*. Mỗi dòng Nlu gồm một ý định và danh sách câu mẫu thuộc ý định đó. Bot học từ các câu mẫu này để nhận ra cả những câu nó chưa gặp bao giờ.

Ví dụ với ý định `ask_working_hours`:

- mấy giờ bên bạn mở cửa
- giờ làm việc thế nào
- thứ 7 có làm việc không
- mấy giờ thì đóng cửa
- chủ nhật có mở không

Viết câu mẫu tốt:

- **Số lượng:** tối thiểu 2 câu, nên 8–15 câu cho mỗi ý định.
- **Đa dạng:** trộn câu dài và ngắn, có dấu và không dấu (“gio lam viec”), thêm cách nói tự nhiên như “ad ơi”, “shop ơi”, “nhé”, “ạ”.
- **Viết như người dùng thật gõ**, không cần đúng chính tả. Nguồn câu tốt nhất là trang **Hội thoại thật** (mục 4.3).
- **Không để một câu nằm ở hai ý định**. Hệ thống sẽ cảnh báo khi phát hiện.
- Ý định nào có quá ít câu mẫu hơn hẳn các ý định khác thì bot sẽ ít khi đoán ra ý định đó.

Đánh dấu thực thể trong câu mẫu: nếu muốn bot lấy ra một thông tin nằm trong câu, bạn bôi thông tin đó theo dạng `[giá trị](tên_thực_thể)`:

- tên mình là `[Thiện](customer_name)`
- số của mình là `[0912345678](phone_number)`

3.3. Thực thể (Entity)

Dùng để làm gì: là *mẫu thông tin cụ thể* bot cần lấy ra từ câu của người dùng, như họ tên, số điện thoại, tên sản phẩm hay ngày giờ.

Ý định cho biết...	Thực thể cho biết...
Người dùng muốn làm gì : <code>inform</code> (đang cung cấp thông tin)	Thông tin cụ thể là gì: <code>customer_name</code> = “Thiện”

Cách dùng:

1. Khai báo thực thể ở trang Thực thể, ví dụ `customer_name`, mô tả “Họ tên khách hàng”.
2. Đánh dấu thực thể trong câu mẫu Nlu, như ví dụ ở mục 3.2. Nên có nhiều giá trị khác nhau để bot học được mẫu chung: `[Thiện]`, `[Nguyễn Văn An]`, `[Trần Thị Bình]` ...
3. Muốn bot *nhớ* giá trị đó để dùng về sau thì tạo thêm một slot (mục 3.4).

3.4. Slots: trí nhớ của bot

Dùng để làm gì: lưu thông tin trong suốt cuộc trò chuyện để dùng lại sau. Thực thể chỉ tồn tại trong câu vừa nhấn. Slot thì giữ giá trị cho các câu sau.

Ví dụ: người dùng nói “tên mình là Thiện”. Bot lưu `customer_name = Thiện`. Mười câu sau người dùng hỏi “tôi tên gì” thì bot vẫn trả lời được: “Bạn tên là Thiện, mình nhớ mà!”

Các ô cần nhập:

Ô	Ý nghĩa
Tên	Tên slot, thường trùng tên thực thể: <code>customer_name</code>
Kiểu	Loại dữ liệu. Hay dùng nhất là <code>text</code> (chữ). Các kiểu khác: <code>bool</code> (có/không), <code>categorical</code> (một trong vài giá trị cố định), <code>float</code> (số), <code>list</code> (danh sách), <code>any</code>
Mapping	Slot lấy giá trị từ đâu. Hay dùng nhất là <code>from_entity</code> : lấy từ thực thể cùng tên

Lưu ý:

- Slot **chưa có mapping** thì sẽ không bao giờ được điền giá trị.
- Kiểu `categorical` phải có danh sách giá trị thì mới ảnh hưởng tới cách bot trả lời.
- Muốn bot trả lời khác nhau tùy slot đã có giá trị hay chưa (như biết tên hay chưa biết tên), bạn làm bằng **Kho hội thoại** (mục 3.7).

3.5. Phản hồi (Response)

Dùng để làm gì: nội dung bot nói ra.

Các ô cần nhập:

Ô	Ý nghĩa	Ví dụ
Tên	Bắt buộc bắt đầu bằng <code>utter_</code>	<code>utter_working_hours</code>
Nội dung	Một hoặc nhiều câu trả lời	“Bên mình làm việc từ 8:00 đến 17:30...”

Mẹo:

- **Nhiều biến thể:** nhập 2–3 câu cùng ý, bot sẽ chọn ngẫu nhiên nên nghe tự nhiên hơn, không lặp lại như máy.
 - “Tạm biệt bạn, hẹn gặp lại!”
 - “Chúc bạn một ngày tốt lành, hẹn gặp lại nhé!”
- **Chèn giá trị slot** bằng dấu ngoặc nhọn: `Rất vui được làm quen với {customer_name}!`

- **Nút bấm:** mỗi nút có *tiêu đề* và *payload*. Payload có dạng `/tên_ý_định`, nên bấm nút giống như người dùng nhấn đúng ý định đó.
 - Nút “Đặt lịch tư vấn” có payload `/request_consultation`
 - Nút “Để sau” có payload `/deny`
- Nếu câu trả lời kết thúc bằng một câu hỏi (“Bạn muốn được tư vấn không?”), bạn cần chuẩn bị luôn kịch bản cho cả hai trường hợp người dùng trả lời *có* và *không* (mục 3.7).

3.6. Rules: luật cố định

Dùng để làm gì: những cặp “**hỏi A thì luôn trả lời B**”, không phụ thuộc vào những gì đã nói trước đó.

Ví dụ:

Tên rule	Người dùng (ý định)	Bot (hành động)
Chào hỏi	<code>greet</code>	<code>utter_greet</code>
Hỏi giờ làm việc	<code>ask_working_hours</code>	<code>utter_working_hours</code>
Người dùng muốn hỏi tiếp	<code>not_finished</code>	<code>utter_continue</code>

Khi nào dùng Rules: câu hỏi đứng một mình, lúc nào cũng trả lời giống nhau, như hỏi giờ, địa chỉ, hotline, chào, cảm ơn. Phần lớn chức năng hỏi–đáp đơn giản chỉ cần rule.

3.7. Kho hội thoại (Stories)

Dùng để làm gì: các **đoạn hội thoại mẫu nhiều lượt**. Bot học từ các đoạn này cách phản ứng *tùy theo ngữ cảnh*.

Ví dụ “Hỏi dịch vụ nhưng chưa cần tư vấn”:

```

người dùng: ask_services      ("bên bạn có dịch vụ gì")
bot:        utter_services    ("Bên mình có... Bạn muốn được tư vấn không?")
người dùng: deny              ("để sau nhé")
bot:        utter_goodbye      ("Tạm biệt bạn, hẹn gặp lại!")
  
```

Trên web, mỗi bước là một ô (node) trong sơ đồ. Có 3 loại bước:

Loại bước	Ý nghĩa
intent	Người dùng nói câu thuộc ý định này. Có thể kèm thực thể.
action	Bot làm việc này. Hiện nay là một phản hồi <code>utter_...</code>
slot_was_set	Lúc này slot đã có giá trị, ví dụ <code>customer_name</code> đã biết

Rules hay Stories?

	Rules	Kho hội thoại (Stories)
Phụ thuộc ngữ cảnh	Không	Có

	Rules	Kho hội thoại (Stories)
Số lượt	1 hỏi, 1 đáp	Nhiều lượt
Bot học như thế nào	Làm đúng y hệt	<i>Suy rộng</i> từ ví dụ ra các tình huống tương tự
Dùng cho	Hỏi–đáp đơn giản	Luồng nhiều bước: đặt lịch, làm quen, xác nhận có/không

Cẩn thận khi bot suy rộng. Ví dụ story trên dạy bot rằng “*deny*” thì chào tạm biệt. Sau đó người dùng nói “không” trong một ngữ cảnh khác, bot cũng chào tạm biệt, dù người dùng chưa muốn dừng. Khi thấy bot suy rộng sai như vậy, bạn thêm story cho đúng ngữ cảnh đó để bot phân biệt được.

Mẹo viết stories:

- Mỗi câu hỏi có/không của bot nên có ít nhất 2 story: một cho có (*affirm*), một cho *không* (*deny*).
- Dùng `slot_was_set` để tách hai nhánh “đã biết” và “chưa biết”. Ví dụ khi người dùng hỏi “tôi tên gì”: nếu đã biết tên thì trả lời tên, nếu chưa biết thì hỏi lại.
- Tên story nên mô tả tình huống: “Đặt lịch tư vấn - hủy”, “Làm quen - hỏi tên khi bot chưa biết”.

4. Nhóm Train: huấn luyện và theo dõi bot

4.1. Train Model

Chức năng	Dùng để làm gì
Kiểm tra dữ liệu	Rà lỗi mà <i>không</i> train. Nên bấm trước mỗi lần train.
Train	Cho bot học dữ liệu mới. Tiến độ hiện theo từng bước: <i>Hàng đợi</i> → <i>Kiểm tra dữ liệu</i> → <i>Train</i> → <i>Nạp model</i> → <i>Hoàn tất</i>
Model đang chạy	Model bot đang dùng để trả lời người dùng
Phiên bản model	Danh sách các lần train. Nếu model mới chạy tệ hơn, bạn chọn một bản cũ để quay lại ngay, không cần train lại

Lưu ý:

- Mỗi lúc chỉ chạy được một phiên train.
- Nếu dữ liệu không thay đổi kể từ lần train trước, hệ thống báo “*Dữ liệu không thay đổi*” và không train lại.
- Train có thể mất vài phút. Nếu chỉ sửa Rules/Stories hoặc Phản hồi mà không sửa câu mẫu, lần train thường nhanh hơn.

4.2. Chat thử

Nơi nói chuyện thử với bot trước khi cho người dùng thật dùng. Với mỗi câu bạn gõ, trang hiện **bot đã hiểu gì**:

- **Ý định và độ tin cậy.** Màu cho biết mức độ chắc chắn:
 - **● từ 80% trở lên:** bot chắc chắn
 - **● 50–80%:** bot tạm chắc, nên thêm câu mẫu
 - **● dưới 50%,** hoặc `nlu_fallback` : bot không hiểu, cần thêm câu mẫu
- **Xếp hạng ý định:** các ý định bot đã cân nhắc. Hai ý định đầu chỉ cách nhau dưới 10% thì bot dễ nhầm giữa chúng. Bạn nên làm câu mẫu của hai ý định đó khác nhau rõ hơn.
- **Thực thể** bot lấy được, **slot được gán**, và **hành động bot đã chọn**.

`nlu_fallback` nghĩa là bot *không đủ tự tin* để chọn ý định nào.

4.3. Hội thoại thật

Toàn bộ tin nhắn người dùng thật gửi qua các kênh (hiện có Telegram). Đây là **nguồn câu mẫu tốt nhất**, vì đó đúng là cách người dùng thật nói.

Tab “Câu cần xem lại” tự gom các tin nhắn đáng chú ý:

Lý do	Nghĩa là
Độ tin cậy thấp	Bot đoán ý định nhưng chắc chắn dưới 60%
Bot không hiểu	Bot trả về <code>nlu_fallback</code>
Bị đánh dấu sai	Có người đã bấm Bot hiểu sai cho câu này khi đọc hội thoại

Với mỗi tin nhắn, bạn có thể:

- **Thêm vào ý định:** chọn ý định đúng, câu sẽ được thêm vào câu mẫu (NLU) của ý định đó. Ô chọn cho tìm theo tên tiếng Việt, gõ không dấu cũng được, hoặc tìm theo mã ý định. Nếu câu đang là câu mẫu của một ý định khác, hệ thống sẽ báo để tránh gây nhầm.
- **Bỏ qua:** câu không cần xử lý.
- **Xem hội thoại:** đọc cả đoạn chat để hiểu ngữ cảnh trước khi quyết định. Trong đoạn chat, bấm **Bot hiểu sai** ở câu nào thì câu đó được đưa vào danh sách cần xem lại.

Sau khi thêm câu mẫu, bạn nhớ **train lại** thì bot mới học được.

Nếu không có ý định nào phù hợp với câu (như trường hợp “tôi chưa hỏi mà”), bạn đừng cố nhét vào ý định gần nhất. Hãy tạo ý định mới theo các bước ở mục 2.

4.4. Lịch sử train

Danh sách các lần train trước: thời gian, kết quả, thời lượng, và lỗi nếu có.

5. Khi bot trả lời bằng câu báo lỗi

Các câu dưới đây **không đến từ dữ liệu bạn nhập**. Chúng được viết sẵn trong hệ thống để báo sự cố:

Bot nói	Nguyên nhân	Cách xử lý
“Xin lỗi, bot đang gặp sự cố. Bạn thử lại sau ít phút nhé.”	Hệ thống không kết nối được tới Rasa (máy chủ bot đang tắt hoặc quá tải)	Báo kỹ thuật viên kiểm tra Rasa. Không liên quan tới dữ liệu train.
“Xin lỗi, mình chưa trả lời được câu này. Bạn thử hỏi cách khác nhé.”	Rasa đang chạy nhưng không trả về câu nào: chưa có model, hoặc ý định chưa được nối tới câu trả lời nào	Kiểm tra ý định đó đã có rule hoặc story chưa, rồi train lại
“Hiện mình chỉ đọc được tin nhắn dạng chữ thôi...”	Người dùng gửi ảnh, sticker hoặc file	Bình thường, không cần xử lý

6. Lỗi và cảnh báo khi kiểm tra dữ liệu

Nút **Kiểm tra dữ liệu** chia kết quả làm hai loại:

- **Lỗi (đỏ):** phải sửa, nếu không sẽ không train được.
- **Cảnh báo (vàng):** vẫn train được, nhưng bot có thể chạy không tốt.

Thông báo	Cách sửa
Ý định “...” chưa có câu mẫu nào	Thêm câu mẫu cho ý định đó ở trang Nlu
Ý định “...” chỉ có 1 câu mẫu	Thêm câu mẫu, nên từ 8 câu trở lên
Câu mẫu “...” xuất hiện ở cả “A” và “B”	Xóa câu đó khỏi một trong hai ý định
Ý định “...” có dữ liệu NLU nhưng chưa được khai báo	Tạo ý định cùng mã ở trang Ý định
... dùng hành động “...” không tồn tại	Tạo phản hồi <code>utter_...</code> cùng tên, hoặc sửa lại tên trong rule/story cho đúng
... dùng ý định “...” chưa được khai báo	Tạo ý định đó, hoặc sửa lại tên trong rule/story
Tên phản hồi “...” nên bắt đầu bằng “utter_”	Đổi tên phản hồi
Slot “...” chưa có mapping nên sẽ không bao giờ được điền	Thêm mapping, thường là <code>from_entity</code>
Slot “...” dùng thực thể “...” chưa được khai báo	Tạo thực thể đó ở trang Thực thể
Câu mẫu “...” gán thực thể “...” chưa được khai báo	Tạo thực thể, hoặc sửa lại chỗ đánh dấu <code>...</code>
... “...” bị trùng tên	Đặt tên khác cho rule/story
Cần ít nhất 2 ý định có câu mẫu để train	Bot cần tối thiểu 2 ý định có câu mẫu

7. Quy trình cải thiện bot hằng tuần

1. Mở **Hội thoại thật**, tab *Cần xem lại*.
2. Với mỗi tin nhắn: thêm vào đúng ý định, tạo ý định mới nếu chưa có, hoặc bỏ qua.
3. Nếu bot trả lời sai ngữ cảnh (dù hiểu đúng ý định), sửa hoặc thêm story trong **Kho hội thoại**.
4. **Kiểm tra dữ liệu**, sửa hết lỗi, rồi **Train**.
5. Vào **Chat thử**, gõ lại vài câu vừa thêm để chắc bot đã hiểu.
6. Nếu model mới chạy tệ hơn, vào **Phiên bản model** để quay lại bản trước.

8. Bảng thuật ngữ

Thuật ngữ	Nghĩa
Ý định (intent)	Mục đích của câu người dùng nói
Câu mẫu (example)	Câu ví dụ để bot học một ý định
NLU	Phần giúp bot <i>hiểu</i> câu: đoán ý định và tìm thực thể
Thực thể (entity)	Mẫu thông tin cụ thể trong câu (tên, số điện thoại...)
Slot	Trí nhớ của bot trong cuộc trò chuyện
Phản hồi (response, utter_...)	Câu bot nói ra
Hành động (action)	Việc bot làm ở một lượt. Hiện nay là gửi một phản hồi.
Rule	Luật cố định: hỏi A thì luôn trả lời B
Story	Đoạn hội thoại mẫu nhiều lượt để bot học theo ngữ cảnh
Train	Cho bot học dữ liệu mới, tạo ra một model mới
Model	Kết quả sau khi train. Bot dùng model để hiểu và trả lời.
Độ tin cậy (confidence)	Bot chắc chắn bao nhiêu phần trăm về ý định nó đoán
<code>nlu_fallback</code>	Bot không đủ tự tin để chọn ý định nào
Payload	Nội dung gửi đi khi bấm nút, dạng <code>/tên_ý_định</code>